

PROJECT PROPOSAL

SaaS-Based Business Management System

Field	Details
Course	CSE 4104 – Software Development III
Project Title	SaaS-Based Business Management System
Submission Date	May 25, 2026

Executive Summary

This proposal presents the design and development plan for a comprehensive Software-as-a-Service (SaaS) Business Management System — a cloud-native, multi-tenant web application built specifically for small and medium-sized enterprises (SMEs). The platform consolidates the most critical business operations into a single, unified interface: human resources, finance, inventory, customer relationships, and project management.

Today's SMEs are forced to juggle multiple disconnected software tools to manage operations, leading to data silos, wasted time, and high cumulative software costs. Our proposed solution eliminates this fragmentation by providing one integrated, subscription-based platform that every department can use simultaneously — with role-based access ensuring each user sees only what they need.

Built on a modern JavaScript stack (React.js, Node.js, PostgreSQL) and deployed on AWS, the system follows a subscription monetization model with tiered plans for businesses of varying sizes. The project will be delivered over 12 weeks using Agile Scrum methodology, producing a fully deployed application, complete documentation, and a GitHub repository.

1. Problem Statement

1.1 Background & Core Problems

The average SME uses 3–8 separate software tools to manage daily operations. This fragmentation creates four critical problems:

Data Fragmentation

Different departments use separate tools with no single source of truth. Employee records in HR do not sync with payroll in finance; inventory is not connected to sales data; project costs are invisible to the finance team. Decision-makers receive conflicting numbers and cannot trust their data.

High Total Software Cost

A typical SME pays separately for HR software (\$50–\$150/mo), accounting (\$30–\$100/mo), CRM (\$50–\$150/mo), project management (\$20–\$80/mo), and inventory (\$30–\$100/mo) — totalling \$180–\$580 per month in software fees alone.

No Real-Time Business Visibility

With data spread across platforms, owners cannot get a real-time view of business performance. They must manually compile reports from different systems — a slow, error-prone process.

Poor Scalability

Most small-business tools are not designed to grow. As a business expands, these tools become inadequate and businesses face disruptive, expensive migrations.

1.2 Why Existing Solutions Fall Short

- Enterprise ERP (SAP, Oracle): Costs tens of thousands of dollars, requires months of implementation — completely beyond SME reach
- Free tools (Google Sheets, Trello): Lack automation, reporting depth, role-based access, and integrations
- Single-purpose SaaS tools: Solve only one problem and create the fragmentation described above
- Open-source ERP (Odoo, ERPNext): Complex to deploy, requires technical expertise, poor modern UX

There is a clear, underserved market for an affordable, all-in-one SaaS platform built specifically for SMEs.

2. Objectives

2.1 Primary Objectives

1. Design and develop a fully functional, production-ready multi-tenant SaaS web application
2. Implement secure role-based authentication with three access levels: Super Admin, Org Admin, and Staff
3. Build and integrate five core modules: HR, Finance, Inventory, CRM, and Project Management
4. Implement a real-time dashboard with KPI metrics and data visualization charts
5. Integrate a complete subscription and payment system using Stripe API
6. Deploy on cloud infrastructure with HTTPS, custom domain, and environment configuration
7. Achieve at least 70% test coverage on all backend API endpoints

2.2 Secondary Objectives

- Build a mobile-responsive frontend accessible on any device
- Implement real-time in-app notifications via WebSockets
- Provide full API documentation via Swagger/OpenAPI
- Enable data export (CSV/PDF) for all major reports
- Set up CI/CD pipeline via GitHub Actions for automated testing and deployment

3. Features & Functional Requirements

3.1 Authentication & User Management

A secure, standards-compliant authentication system supporting email/password and OAuth social login (Google). Sessions managed using short-lived JWT access tokens with rotating refresh tokens.

- User registration with email verification and password reset
- OAuth 2.0 social login (Google)
- Role-based access control (RBAC): Super Admin, Org Admin, Staff

- Two-factor authentication (2FA) for admin accounts
- Complete audit logs tracking all user actions

3.2 Dashboard & Analytics

A personalized, role-specific dashboard aggregating data from all modules in real-time visual format. Managers see operational overviews; staff see personal task lists and attendance.

- KPI cards: revenue, expenses, employee count, open tickets, inventory value
- Sales and revenue trend charts (line, bar, pie) using Chart.js
- Upcoming tasks, deadlines, low-stock alerts, and overdue invoice notifications
- Date range filtering on all reports and data export to PDF/CSV

3.3 Human Resource Management (HRM)

Central repository for all employee data — managing the complete employee lifecycle from onboarding to offboarding with automated payroll and leave management.

- Employee profiles: personal info, job title, department, contract type, salary
- Attendance tracking with clock-in/out and daily summary reports
- Leave application, approval workflow, and balance tracking
- Automated payroll calculation with configurable deductions (tax, insurance)
- Payslip generation and bulk download as PDF
- Performance review scheduling and document management

3.4 Inventory Management

Complete visibility into stock levels, supplier relationships, and product movement — preventing stockouts through automated alerts and easy purchase order creation.

- Product catalog with categories, SKUs, units of measure, and images
- Real-time stock level tracking with low-stock threshold alerts
- Purchase order creation and supplier management with GRN workflow
- Inventory valuation report (FIFO / weighted average cost)
- Barcode/QR code generation for product labels and full stock movement history

3.5 Finance & Accounting

Handles invoicing, expense tracking, and financial reporting — designed for non-accountants with an intuitive interface that maintains accuracy.

- Invoice creation with line items, tax, and discount support
- Recurring invoice automation and payment recording
- Expense tracking with categories and receipt uploads
- Profit & Loss (P&L) statement and accounts payable/receivable dashboards
- Multi-currency support and budget vs. actual variance tracking

3.6 Customer Relationship Management (CRM)

Tracks every interaction with leads and customers, enabling effective pipeline management and sales performance visibility.

- Contact and company profiles with full history
- Lead pipeline with Kanban view: New → Contacted → Qualified → Won/Lost
- Activity logging: calls, emails, meetings, follow-up reminders
- Sales performance reports: conversion rates, deal values, rep leaderboards

3.7 Project Management

Helps teams plan, execute, and monitor projects with flexible Kanban board as primary view plus list and timeline (Gantt) options.

- Task boards: Kanban view (To Do / In Progress / Review / Done)
- Sub-tasks, dependencies, time tracking per task and per project
- Milestone tracking with on-track, at-risk, and delayed indicators
- File attachments and inline comments; integration with HR for team availability

3.8 SaaS Platform Features

Features unique to the multi-tenant SaaS architecture:

- Tenant isolation: each organization's data fully separated at the database level
- Custom subdomain per tenant (e.g., companyname.platform.com)
- Self-service subscription management: upgrade, downgrade, cancel
- Super Admin panel for platform-level tenant and billing management
- Feature flags enforcing per-tier access controls and usage quotas

4. Technology Stack

4.1 Selected Technologies

Technologies selected based on developer productivity, scalability, and industry adoption. All are open-source with large communities and direct industry relevance.

Layer	Technology	Purpose
Frontend	React.js / Next.js	Component-based UI with server-side rendering
Styling	Tailwind CSS	Utility-first responsive design system
Backend	Node.js / Express.js	RESTful API server and business logic
Database	PostgreSQL	ACID-compliant relational data storage
ORM	Prisma ORM	Type-safe DB access and schema migrations
Auth	JWT / OAuth 2.0	Stateless tokens and social login
File Storage	AWS S3	Scalable document and media storage
Cloud Hosting	AWS EC2 / Vercel	Auto-scaling cloud deployment
Payment	Stripe API	PCI-compliant subscription billing
Real-time	Socket.io	WebSocket events for live notifications
Version Control	Git / GitHub	Branching, CI/CD pipelines, code review

4.2 Architecture Overview

The system follows a three-tier architecture: (1) Presentation Tier — React.js/Next.js SPA communicating via REST APIs; (2) Application Tier — Node.js/Express.js handling business logic and integrations; (3) Data Tier — PostgreSQL with row-level security for tenant isolation, managed via Prisma ORM.

4.3 Multi-Tenant Data Strategy

A shared database, shared schema approach with a `tenant_id` foreign key on all tables. PostgreSQL row-level security (RLS) policies automatically filter every query by tenant, preventing cross-tenant data leakage. This allows one schema to migrate and one database to back up, while providing strong logical isolation.

4.4 Security Architecture

- HTTPS enforced everywhere with HSTS headers; passwords hashed with bcrypt (factor 12)
- JWT access tokens expire in 15 minutes; refresh tokens rotate on every use
- Input validation and sanitization on all endpoints against SQL injection and XSS
- Rate limiting on auth endpoints to prevent brute-force attacks
- CORS configured to allow only known frontend origins; secrets via AWS Secrets Manager

5. Subscription & Monetization Model

5.1 Pricing Tiers

Three subscription tiers cater to businesses at different growth stages. Billing is handled entirely by Stripe, which manages payment processing, recurring charges, and invoicing.

Feature	Free	Pro (\$29/mo)	Enterprise
Users	Up to 3	Up to 25	Unlimited
Storage	500 MB	20 GB	Unlimited
HR Module	Basic	Full	Full + Custom
Finance Module	No	Yes	Yes + Reports
API Access	No	Yes	Yes + Webhooks
Support	Community	Email	Dedicated Manager

5.2 Billing Features

- 14-day free trial for all paid plans — no credit card required at signup
- Monthly and annual billing (annual = 2 months free)
- Automated invoice emails; failed payment retry with dunning logic (3 attempts over 7 days)
- Self-service billing portal for subscription and payment method management

6. System Design

6.1 Database Design Principles

- UUID v4 primary keys to prevent enumeration attacks
- Soft deletes using `deleted_at` timestamp — preserving full audit history
- `created_at` and `updated_at` on every table; `tenant_id` on all business data tables
- Foreign key constraints for referential integrity; composite indexes on `tenant_id` + `created_at`

6.2 API Design

- Resource-based REST URLs (e.g., `GET /api/v1/employees`, `POST /api/v1/invoices`)

- HTTP status codes used correctly: 200, 201, 400, 401, 403, 404, 422, 500
- Consistent JSON response envelope: { success, data, message, pagination }
- API versioning via URL prefix (/api/v1/) for backwards compatibility
- Cursor-based pagination on all list endpoints; full Swagger/OpenAPI documentation

7. Project Planning & Timeline

7.1 Development Methodology

The project follows Agile Scrum with two-week sprints. Each sprint begins with a planning session, includes daily standups, and ends with a review and retrospective. A product backlog is maintained in GitHub Projects and groomed before each sprint. A 15% time buffer is built into the schedule to accommodate unforeseen delays.

7.2 Sprint Milestones

Phase	Timeline	Key Activities	Deliverable
Phase 1	Weeks 1–2	Requirements, system design, DB schema, wireframes	SRS Document
Phase 2	Weeks 3–4	Backend API, authentication, multi-tenant setup	Working API
Phase 3	Weeks 5–7	Frontend dashboard, HR, Inventory, Finance modules	Core UI Modules
Phase 4	Weeks 8–10	CRM, project mgmt, billing, real-time notifications	Full Feature Set
Phase 5	Weeks 11–12	QA testing, bug fixing, deployment, documentation	Final Product

7.3 Sprint 1 & 2 Detailed Breakdown

Sprint 1 (Week 1)

- Finalize requirements and create user stories in GitHub Projects
- Design database schema, create ERD, and set up Docker Compose dev environment
- Create UI wireframes for all major screens in Figma

Sprint 2 (Week 2)

- Set up Next.js frontend and Express.js backend with TypeScript
- Configure PostgreSQL, Prisma ORM, and multi-tenant architecture
- Implement full authentication: register, login, JWT, refresh token, OAuth

7.4 Team Roles & Responsibilities

- Project Manager / Scrum Master – Sprint planning, task tracking, stakeholder communication
- Backend Developer(s) – API, database design, authentication, third-party integrations
- Frontend Developer(s) – UI implementation, component library, state management
- QA Engineer – Test case writing, manual and automated test implementation

8. Risk Analysis & Management

8.1 Risk Register

Risk	Likelihood	Impact	Mitigation
Scope creep	High	High	Strict sprint planning and backlog control
DB performance under load	Medium	High	Indexing, query optimization, Redis cache
Security vulnerabilities	Medium	Critical	OWASP practices, regular security reviews
Third-party API downtime	Low	High	Graceful error handling and fallback logic
Team member availability	Medium	Medium	Task documentation and knowledge sharing

8.2 Mitigation Approach

The team will conduct a brief risk review at the start of each sprint. Any risk that materializes is escalated to the supervisor within 24 hours. General mitigations include: code reviews on every PR, weekly backups of all data, clear documentation of all decisions, and a 15% schedule buffer to absorb delays.

9. Deliverables

9.1 Required Submissions

8. Project Proposal Document — This document (submitted as PDF)
9. GitHub Repository — Complete source code with README and setup guide
10. Team Information Sheet — Names, student IDs, roles, and contacts
11. Initial Project Plan — Sprint schedule, user story backlog, GitHub Projects board
12. Final Working Application — Deployed and publicly accessible via URL
13. Technical Documentation — API docs (Swagger), ERD, architecture diagrams
14. Test Report — Coverage summary, test cases, and results

15. Final Presentation — Live demo and Q&A with the project supervisor

9.2 Documentation Standards

All code must include inline comments for complex logic. The README must enable a new developer to run the project locally within 15 minutes. API documentation must be complete enough for a frontend developer to integrate without reading source code.

10. Conclusion

The SaaS-Based Business Management System addresses a real and significant challenge faced by SMEs worldwide. By consolidating fragmented business operations into a single, affordable, cloud-native platform, this project delivers genuine value to its target users while demonstrating end-to-end software development competency.

From a technical perspective, the project provides an excellent opportunity to apply skills in full-stack web development, cloud architecture, database design, API integration, and secure software engineering. The modern, industry-relevant technology stack will provide the team with skills directly transferable to the professional market.

The team is committed to delivering a polished, functional product that meets all CSE 4104 requirements and serves as a portfolio-worthy demonstration of professional software development. With disciplined Agile execution and the technical architecture outlined in this proposal, we are confident in delivering all stated objectives within the 12-week timeline.

We respectfully request approval of this proposal to proceed with full development.